🧪 Lab Manual: Refactoring Using SOLID Principles (Java)

🎯 Objective

Refactor an existing system step-by-step to apply all five SOLID principles using an evolving codebase.

🧩 Scenario: E-Commerce Order Processing System

The system performs:

- Order total calculation
- Discount handling
- Payment processing
- Shipping handling
- Customer notification

The current implementation is tightly coupled and violates SOLID principles.

📄 Initial Code (Given)

```
class OrderProcessor {

    public double processOrder(String[][] order, String paymentType, String shippingType) {
        double total = 0;

        // Calculate total
        for (String[] item : order) {
            total += Double.parseDouble(item[1]);
        }

        // Apply discount
        if (total > 100) {
            total *= 0.9;
        }

        // Payment processing
        if (paymentType.equals("credit")) {
            System.out.println("Processing credit card payment");
        } else if (paymentType.equals("paypal")) {
            System.out.println("Processing PayPal payment");
        }

        // Shipping
        if (shippingType.equals("standard")) {
            System.out.println("Standard shipping selected");
        } else if (shippingType.equals("express")) {
            System.out.println("Express shipping selected");
        }

        // Notification
        System.out.println("Sending email notification");

        return total;
    }
}
```

🛠 Tasks

🔹 Task 1: Single Responsibility Principle (SRP)

Refactor the system so that each class has only one responsibility. Identify different responsibilities in the current class and separate them into multiple classes. Ensure each class performs a single well-defined task.

Hint (Bad Design to Include): Keep at least one class where multiple responsibilities are still mixed (e.g., calculation + discount + printing). Then refactor it properly.

Task 2: Open/Closed Principle (OCP)

Make the system open for extension but closed for modification. Remove conditional logic used for payment and shipping. Introduce abstractions (interfaces or abstract classes) and design the system so new features can be added without modifying existing code.

Hint (Bad Design to Include): Add a new payment type (e.g., `bkash`) by modifying existing `if/else` logic. Then refactor to remove conditionals using polymorphism.

Task 3: Liskov Substitution Principle (LSP)

Ensure that derived classes can replace base classes without breaking functionality. Create an implementation that violates expected behavior, analyze the issue, and refactor the design so all implementations behave consistently.

Hint (Bad Design to Include): Create a payment class that throws an exception or restricts valid inputs (e.g., fails for large amounts). Then fix the design so all implementations follow the same contract.

Task 4: Interface Segregation Principle (ISP)

Avoid forcing classes to implement methods they do not use. Identify large interfaces and split them into smaller, more specific ones so that classes only implement what they require.

Hint (Bad Design to Include): Create a single interface with multiple methods (e.g., `pay()` and `refund()`) and force all classes to implement both, even if not needed. Then refactor into smaller interfaces.

Task 5: Dependency Inversion Principle (DIP)

Refactor the system to depend on abstractions rather than concrete implementations. Introduce interfaces where necessary and apply dependency injection to decouple high-level and low-level modules.

Hint (Bad Design to Include): Instantiate concrete classes directly inside the main processor (e.g., `new CreditCardPayment()`). Then refactor to use interfaces and constructor injection.

⚙ Constraints

- Do not modify the core structure unnecessarily
- Do not hardcode new logic inside the main processor
- Focus on modular design and clean architecture
- Use proper object-oriented principles
- Maintain readable and well-structured code